

PLSQL_Exercises

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

Creation :

```
[ SQL Worksheet ]*  ▶  ⌵  🔑  📄  ⌵  Aa  🗑️

1  CREATE TABLE Customers1 (
2      CustomerID NUMBER PRIMARY KEY,
3      CustomerName VARCHAR2(50),
4      Age NUMBER,
5      Balance NUMBER,
6      IsVIP VARCHAR2(10)
7  );
8
9  CREATE TABLE Loans1 (
10     LoanID NUMBER PRIMARY KEY,
11     CustomerID NUMBER,
12     InterestRate NUMBER,
13     DueDate DATE
14 );|
```

Insertion:

```
[ SQL Worksheet ]*  ▶  ⌵  🔑  📄  ⌵  Aa  🗑️

1  INSERT INTO Customers1 VALUES (101, 'Ravi', 65, 15000, 'FALSE');
2  INSERT INTO Customers1 VALUES (102, 'Priya', 45, 8000, 'FALSE');
3  INSERT INTO Customers1 VALUES (103, 'Kiran', 70, 20000, 'FALSE');
4
5  INSERT INTO Loans1 VALUES (5001, 101, 8, SYSDATE + 10);
6  INSERT INTO Loans1 VALUES (5002, 102, 9, SYSDATE + 40);
7  INSERT INTO Loans1 VALUES (5003, 103, 7, SYSDATE + 20);
8
9  COMMIT;
```

Question: Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

```

[ SQL Worksheet ]*  ▶  ⌵  🔑  🔄  ⌵  Aa  🗑️

1 DECLARE
2     CURSOR cust_cursor IS
3         SELECT CustomerID, Age
4         FROM Customers1;
5 BEGIN
6     FOR cust IN cust_cursor LOOP
7         IF cust.Age > 60 THEN
8             UPDATE Loans1
9             SET InterestRate = InterestRate - 1
10            WHERE CustomerID = cust.CustomerID;
11        END IF;
12    END LOOP;
13
14    COMMIT;
15 END;
16 /
17 select * from LOANS1;

```

OUTPUT :

	LOANID	CUSTOMERID	INTERESTRATE	DUEDATE
1	5001	101	7	7/1/2026, 11:16:33
2	5002	102	9	7/31/2026, 11:16:33
3	5003	103	6	7/11/2026, 11:16:33

Scenario 2: Manage errors when updating employee salaries.

Question: Write a stored procedure **UpdateSalary** that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

```

[ SQL Worksheet ]*  ▶  ⌵  🔑  🔄  ⌵  Aa  🗑️

1 DECLARE
2     CURSOR cust_cursor IS
3         SELECT CustomerID, Balance
4         FROM Customers1;
5 BEGIN
6     FOR cust IN cust_cursor LOOP
7         IF cust.Balance > 10000 THEN
8             UPDATE Customers1
9             SET IsVIP = 'TRUE'
10            WHERE CustomerID = cust.CustomerID;
11        END IF;
12    END LOOP;
13
14    COMMIT;
15 END;
16 /
17 SELECT * FROM Customers1;

```

OUTPUT:

	CUSTOMERID	CUSTOMERNAME	AGE	BALANCE	ISVIP
1	101	Ravi	65	15000	TRUE
2	102	Priya	45	8000	FALSE
3	103	Kiran	70	20000	TRUE

Scenario 3: Ensure data integrity when adding a new customer.

Question: Write a stored procedure **AddNewCustomer** that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

```
[ SQL Worksheet ]*  ▶  ⌵  🔑  📄  ⌵  Aa  🗑️

1  BEGIN
2      FOR loan IN (
3          SELECT CustomerID, LoanID, DueDate
4          FROM Loans1
5          WHERE DueDate BETWEEN SYSDATE AND SYSDATE + 30
6      )
7      LOOP
8          DBMS_OUTPUT.PUT_LINE(
9              'Reminder: Customer ' || loan.CustomerID ||
10             ' has Loan ' || loan.LoanID ||
11             ' due on ' || TO_CHAR(loan.DueDate, 'DD-MON-YYYY')
12          );
13      END LOOP;
14  END;
15  /
```

OUTPUT:

Query result	Script output	DBMS output	Explain Plan	SQL history
 				
Reminder: Customer 101 has Loan 5001 due on 01-JUL-2026 Reminder: Customer 103 has Loan 5003 due on 11-JUL-2026				
PL/SQL procedure successfully completed.				